# Bachelor Thesis

Computer Science and Engineering 300 credits

# Design and Implementation of an Embedded System for Real-Time Condition-Based Monitoring and Anomaly Detection

Computer Science and Engineering 15 credits

Halmstad June 7, 2026

Axel Ekstrand

Pontus Nestor

HALMSTAD UNIVERSITY

# Abstract

This project presents the design and implementation of an embedded system for real-time data acquisition and handling from sensors in an industrial environment. The system was designed to collect, process, and visualise data from multiple sensors using IO-Link communication. The system consists of a Raspberry Pi and an IO-Link master, with the MQTT protocol used for communication. Sensor data was sent from the IO-Link master to the Raspberry Pi for processing. This was done using statistical methods such as rolling averages and standard deviations for anomaly detection. The data was then presented to the user using a graphical user interface. This enabled real-time monitoring and alert generation.

The results show that the constructed system successfully handled continuous data streams of two data points per second from the three sensors. The system also detected deviations introduced during testing by detecting values that deviated from the norm and irregular behaviour. The artefact operates independently of the existing hardware and does not have a cloud service connection.

This project demonstrates an approach to IoT-based condition monitoring and presents the potential for future development towards scalable data storage and predictive maintenance for the system.

**Keywords** Embedded System, Condition-Based Monitoring, MQTT, IO-Link, Sensors, Graphical User Interface, Python

# Sammanfattning

Detta projekt presenterar design och implementering av ett inbyggt system för realtidsdatainsamling och hantering från sensorer i en industriell miljö. Systemet utformades för att samla in, bearbeta och visualisera data från flera sensorer via IO-Link-kommunikation.

Systemet består av en Raspberry Pi och en IO-Link-master, med MQTT som kommunikationsprotokoll. Sensordata skickades från IO-Link-mastern till Raspberry Pi för bearbetning. Detta gjordes med hjälp av statistiska metoder, som rullande medelvärden och standardavvikelser, för att mäta avvikelsen från normaliteten. Data presenterades sedan för användaren via ett grafiskt användargränssnitt. Detta möjliggjorde realtidsövervakning och generering av varningar.

Resultaten visar att det framtagna systemet framgångsrikt hanterade kontinuerliga dataströmmar med två datapunkter per sekund från de tre sensorerna. Systemet upptäckte också avvikelser som introducerades under testning genom att detektera värden som avvek från normen och oregelbundet beteende. Artefakten fungerar oberoende av befintlig hårdvara och har ingen anslutning till någon molntjänst.

Detta projekt demonstrerar en metod för IoT-baserad tillståndsövervakning och presenterar potentialen för framtida utveckling mot skalbar datalagring och prediktivt underhåll av systemet.

**Nyckelord** Embedded System, Condition-Based Monitoring, MQTT, IO-Link, Sensors, Graphical User Interface, Python

# Preface

This report presents the development of an embedded system for real-time condition-based monitoring, including sensor data analysis, anomaly detection, and a user interface. The project was carried out collaboratively, where we both contributed to the system design, the implementation of hardware and software components, testing, and report writing.

We would like to thank our supervisors, Kevin Hernández Diaz and Fernando Alonso-Fernandez, for their guidance and feedback throughout the project and for providing the Raspberry Pi. We would also like to thank the company Core Link and its R&D Manager, Nils Strandh, for enabling this thesis by defining the project and providing the necessary sensors and equipment for testing our system. The project was carried out at Halmstad University.

# Contents

**Abstract** . . . . . **iii**
**Sammanfattning** . . . . . **v**
**Preface** . . . . . **vii**
**Contents** . . . . . **ix**
**List of Figures** . . . . . **xi**
**Listings** . . . . . **xiii**
**1 Introduction** . . . . . **1**
1.1 Background . . . . . 1
1.2 Problem Statement . . . . . 2
1.3 Purpose and goals . . . . . 3
1.4 Requirements . . . . . 3
1.5 Related works . . . . . 6
1.5.1 Predictive Maintenance in Industry 4.0 . . . . . 6
1.5.2 Smart Sensors and Industrial Monitoring . . . . . 7
1.5.3 Summary of Related Work . . . . . 7
**2 Technical Background** . . . . . **9**
2.1 Embedded Systems . . . . . 9
2.2 Data Transmission with MQTT . . . . . 9
2.3 Smart Sensor and Condition-Based Maintenance . . . . . 10
**3 Method** . . . . . **13**
3.1 Development Process . . . . . 13
3.1.1 Hardware and System Design . . . . . 13
3.1.2 Communication and Architecture . . . . . 14
3.1.3 Data Processing and Analysis . . . . . 14
3.1.4 Software Engineering Practices . . . . . 15
3.2 Technical Design . . . . . 15
3.3 Testing . . . . . 17
3.3.1 Unit testing . . . . . 17
3.3.2 Integration testing . . . . . 17
3.3.3 System testing . . . . . 17
**4 Implementation** . . . . . **21**
4.1 System Setup . . . . . 21
4.2 IO-Link Configuration . . . . . 21
4.3 Communication . . . . . 21

4.4 Data Processing . . . . . 22
4.5 Software Implementation . . . . . 23
4.6 User Interface . . . . . 23
**5 Results . . . . . 25**
5.1 Graphical User Interface . . . . . 25
5.2 Tests Under Normal Conditions . . . . . 25
5.3 IMC18 Tests . . . . . 30
5.4 WTM12F Tests . . . . . 31
5.5 WLG12P Tests . . . . . 32
**6 Discussion . . . . . 33**
6.1 Software Performance and Requirement Verification . . . . . 33
6.2 Hardware Mounting Limitations and Baseline Variability . . . . . 33
6.3 Analysis of Sensor Anomalies and Environmental Disturbances . . . 34
6.4 Societal, Ethical, and Privacy Requirements . . . . . 35
6.4.1 Economic Requirements . . . . . 35
6.4.2 Environmental Sustainability . . . . . 35
6.4.3 Workplace Safety . . . . . 36
6.4.4 Integrity and GDPR Compliance . . . . . 36
**7 Conclusion . . . . . 37**
7.1 Future Work . . . . . 37
**Bibliography . . . . . 39**
**A Annex . . . . . 45**
A.1 Archived web sources . . . . . 45
A.1.1 VTech Peek Learn Doll Productpage . . . . . 45

# List of Figures


1.1 A flowchart showing how the data flows in the system. . . . . 2
1.2 Dust covering the sensor's ability to emit light. . . . . 4
1.3 IMC18 sensor rotated away from the arm to simulate misalignment from vibrations. . . . . 4
1.4 Reflex rotated away from the parallel of the machine's edge, to simulate misalignment from vibrations. . . . . 4
1.5 The CL 500 manual cutter used during testing. . . . . 5

3.1 A picture showing the GUI implemented for the system. (1) Datastream, (2) average, (3) standard deviation, (4) sensor status, (5) alerts window, (6) history button . . . . . 16
3.2 Figure of the system setup. (1) WLG12P photoelectric sensor, (2) saw blade, (3) WTM12F photoelectric sensor, (4) IMC18 inductive sensor, (5) safety guard arm. . . . . 18

5.1 The system operating under normal conditions, where each cut remains within the standard deviation limits and no errors have been detected since system startup. . . . . 26
5.2 The system is operating under abnormal conditions, with the IMC18 sensor rotated out of position, causing measured values to exceed the standard deviation limits and triggering multiple alerts. . . . . 27
5.3 Historical view of the WTM12F and IMC18 sensor values during normal operation. The blue line represents the raw sensor values, while the yellow line represents the calculated average. . . . . 28
5.4 Graph showing WTM12F sensor values during normal operating conditions. . . . . 29
5.5 Graph showing WLG12P sensor values during normal operating conditions. . . . . 29
5.6 Graph showing IMC18 sensor values during normal operating conditions. . . . . 30
5.7 Graph showing IMC18 sensor values during the rotation test used to simulate vibration-induced sensor misalignment. . . . . 30
5.8 Graph showing IMC18 sensor values during the tape test, which simulates dust obstructing the sensor surface. . . . . 31

5.9 Graph showing WTM12F sensor values during the rotation test used to simulate vibration-induced sensor misalignment. . . . . 31
5.10 Graph showing WTM12F sensor values during the dust test, which simulated dust accumulation on the sensor surface. . . . . 32
5.11 Graph showing WLG12P sensor values during the rotation test used to simulate vibration-induced sensor misalignment. . . . . 32

# Listings

4.1 Terminal command for data storage . . . . . 22

# Chapter 1

# Introduction

## 1.1 Background

Industry 4.0, also known as the fourth industrial revolution, builds on Industry 3.0, which introduced electronics, computers, and automation into industrial production. Industry 4.0 builds on this foundation by focusing on interconnected, intelligent, autonomous industrial systems.This is achieved through the integration of multiple advanced technologies, including artificial intelligence (AI), Internet of Things (IoT), autonomous robots, and big data, all connected via network protocols [1–3]. The evolution of embedded systems plays a crucial role in enabling these technologies, providing low-power, efficient, and reliable computing platforms that support real-time processing and decision-making [2]. One key application of Industry 4.0 enabled by embedded systems is predictive maintenance (PdM), where condition-monitoring data from IoT devices is analysed to detect anomalies and prevent unexpected downtime [3]. By using real-time sensing and edge computing, embedded systems enable predictive maintenance that is faster, more precise, and more cost-effective.

Predictive maintenance aims to anticipate failures before they occur by monitoring conditions in real time. When used in industrial systems, it reduces system stops, improves system performance, and reduces maintenance costs [4]. It relies on sensor data to capture changes in operating behaviour and identify patterns indicative of wear, degradation, or abnormal operation. By analysing these patterns rather than reacting to single-sensor fault events, predictive maintenance can enable informed maintenance planning for different parts of the system, including the sensors. This allows maintenance actions to be scheduled based on the system's actual condition rather than fixed service intervals or after a system failure.

This report presents the development of an embedded system for sensor condition monitoring and anomaly detection with a graphical user interface. The system and its interface are intended for a potential organisation's customer service. The goal for the system is to process active sensor status to identify signs of degradation and alert on these deviations that could indicate the need for maintenance.

The system consists of three sensors: a photoelectric proximity sensor, a photoelectric retroreflective sensor, and an inductive proximity sensor. The sensors will be connected to an IO-Link master, a point-to-point, digital communication gateway, standardised in IEC61131-9 [5]. The IO-Link master will then be connected to a Raspberry Pi 4 model B which will serve as the central component that processes the data and displays it on a connected screen. The system will be standalone, making it easy to integrate with the company's existing machines and requiring no changes to the original system's software.

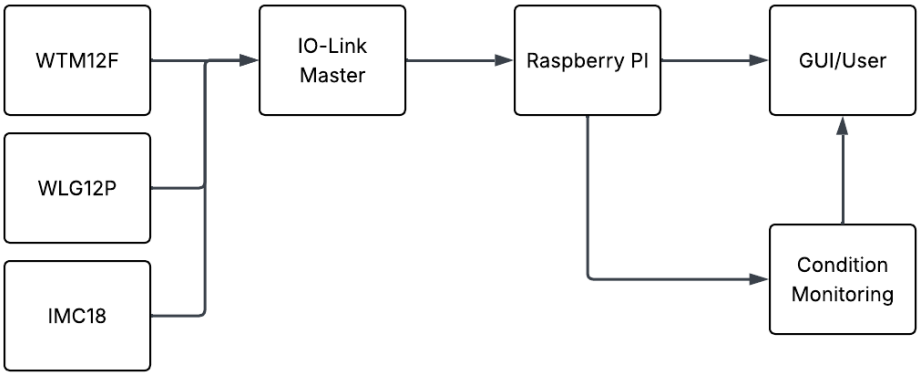



**Figure 1.1:** A flowchart showing how the data flows in the system.

## 1.2 Problem Statement

The challenge and main problem statement for this project was the lack of data from sensors and existing systems. Predictive maintenance and condition-based monitoring traditionally require substantial historical data from sensors and other indicators within a system. To address the lack of data, the project implemented a smart photoelectric proximity sensor and an inductive sensor, both of which provided signal-strength feedback.

The lack of historical data required multiple sensor tests to be conducted in a controlled environment. These tests provided different parameters that could be used as baseline data for working sensors and a working system. Once the baseline parameters had been set, additional tests were performed to determine how the sensors' signal strength and their functionality degrade under different fault types.

This project also faced several technical challenges. The first was to determine how the sensors should be configured to provide the required diagnostic data via an IO-Link master. For example, which bits from the sensor were useful, and the overall configuration of the sensors. The second challenge was to establish a connection between the IO-Link master and the Raspberry Pi to retrieve sensor

data for the system. The IO-Link master provided an Ethernet interface via an M12 X-coded connector, and the Raspberry Pi had an Ethernet port available. This required an understanding of how Ethernet-based communication and protocols work.

To provide a measurable basis for evaluation, two success criteria were defined. First, the system should generate an alert for each fault type introduced during testing. Second, alerts should be generated within one sampling interval of a deviating value being received, i.e., within 0.5 seconds.

## 1.3 Purpose and goals

The purpose of this project was to create a standalone embedded system that could be integrated into an existing system. The system should serve as a condition-monitoring system with possible predictive maintenance capabilities. It should do this by analysing degradation and detecting anomalies via deviations from the standard signal behaviour.

The goal was to design and evaluate a prototype that can detect various degradations in sensor data, warning and informing users about potential problems and anomalies in the sensor function. This prototype was then tested on a manual cutting machine, where its functions were evaluated under different conditions to determine whether it provided accurate and trustworthy responses.

## 1.4 Requirements

The main task of the system was to collect sensor data from different sensors, detect degradation in their signals over time and detect anomalies.

To evaluate the system's functionality, each sensor had fault types defined that the system was intended to handle. For both photoelectric sensors, dust was a fault type. If either the reflector or the sensor were to be covered in dust, the system could return a false status. Because the sensors are often placed inside a machine, if this fault occurred, the actual status could not be observed. To resolve this fault, manual maintenance must be performed, which causes downtime for the active process. This fault type could easily be created in the manual cutting machine to simulate degradation, with varying amounts of the sensor blocked.

Another fault type for both photoelectric sensors was the rotation of the sensor or reflector due to vibration. For the photoelectric reflective sensor, which only worked when the reflector was aligned, misalignment would cause the sensor to stop working. In a photoelectric proximity sensor, misalignment due to vibrations can result in a weaker signal. This fault could also be easily simulated in the cutting machine environment by manually adjusting the reflector and sensor during the test.

For the inductive sensor, the fault type was caused by vibrations that changed the sensor's angle, potentially resulting in a weaker signal. There were two ways

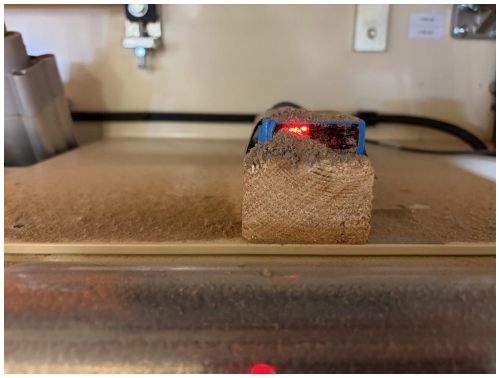


**Figure 1.2:** Dust covering the sensor's ability to emit light.

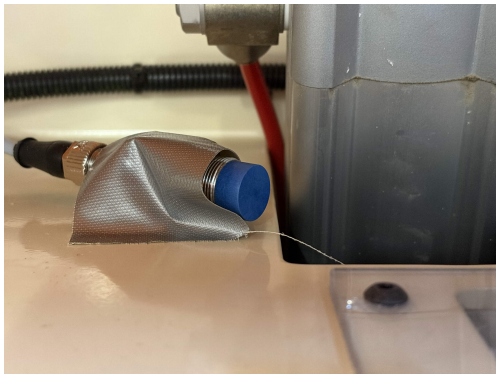


**Figure 1.3:** IMC18 sensor rotated away from the arm to simulate misalignment from vibrations.



**Figure 1.4:** Reflex rotated away from the parallel of the machine's edge, to simulate misalignment from vibrations.

to simulate this: the first was to manually adjust the sensor's angle between tests. The other simulation was to loosen the sensor enough so that it vibrated out of position without danger for the system.




**Figure 1.5:** The CL 500 manual cutter used during testing.

The definition of a test in this project is the process of cutting a core a certain number of times under one of the fault-type conditions. The tests were conducted on a CL 500 manual core cutter, an enclosed cutting unit designed for low noise and high safety. This machine generates vibrations during operation, which can loosen components. It also leaves a layer of dust that can cover sensors. Because the system's goal was to detect patterns over time, each run consisted of about 10 cuts of a core under the same conditions before the conditions changed.

The functional and non-functional requirements of the system are:

Functional requirements:

- The system must be able to acquire and read sensor data from multiple sensors.
- The system is to be able to detect sensor degradation patterns in sensor data.
- The system should present results to a user via an interface to indicate whether the sensors need maintenance.

Non-Functional requirements:

- The system must be testable in a controlled environment with repeatable results.
- The system is to be run in real time without interference with the existing software.
- The system must be able to detect anomalies in sensor data through condition monitoring.
- The system is not allowed to be cloud-connected.

## 1.5 Related works

### 1.5.1 Predictive Maintenance in Industry 4.0

Smart factories are a key component of Industry 4.0, aiming to improve the productivity, flexibility and efficiency of industrial production systems [6]. This is achieved through IoT technologies that connect machinery, sensors, and other industrial components to a central network. Device connectivity enables communication, automated operations, and improved control and monitoring of the industrial processes.

Another important aspect of Industrial IoT is edge computing, which aims to reduce decision-making latency and conserve bandwidth resources. According to [7], edge computing is a distributed computing model that processes data in parts, closer to the data source rather than in a central cloud environment. The article presents various aspects and architectures of edge computing and discusses its implications for smart factories and Industrial IoT systems. The article is relevant to this project, which implements an embedded system with local data processing close to the data source and real-time analysis.

The review article [6] describes several technologies and components used in smart factories in Industry 4.0 from an IoT perspective. Relevant components to this project include advancements in sensor technology, predictive maintenance, real-time system monitoring and analysis.

The article [4] provides a comprehensive overview of predictive maintenance in Industry 4.0. It discusses the typical workflow for predictive maintenance projects and describes the main models and methods available. These methods include Condition-Based Maintenance (CBM), Prognostics and Health Management (PHM), and Remaining Useful Life (RUL) estimation. Among these, the Condition-Based maintenance method is best applicable to this project. This method bases decisions on the equipment's current condition, provided by the sensors in this project.

The CBM and PHM maintenance approaches and the architecture of such a system are explained more in depth in the paper [8]. The paper describes the architecture of an intelligent predictive maintenance system that integrates machine learning with CBM and PHM to provide guided intelligent decision support. The system architecture was divided into six functional blocks, following the Open System Architecture for Condition-Based Maintenance from MIMOSA [9], a standard architecture for CBM systems for moving information. These functional blocks influenced the system architecture used in this project, which focuses on acquiring, processing, analysing, and detecting degradation through sensor data.

In the article [10], a new self-diagnostic system for monitoring the different states of a traffic light was developed. Because standard condition-monitoring methods are not applicable to a traffic light, the authors used sensor nodes to monitor the state of the different lamps. The sensor data were then compared with the expected output to identify faults. This approach aligns with the system implemented in this project, which involves multiple sensors collaborating within a single system. If sensors expected to be in the same state are in different states,

the system should alert to a potential problem.

### 1.5.2 Smart Sensors and Industrial Monitoring

Recent studies have highlighted the growing use of smart sensors in industrial monitoring systems [11]. These sensors monitor system conditions, including vibration, temperature, and position. Intelligent sensors combined with IoT technologies enable real-time data collection and communication between industrial devices and monitoring systems. The literature further describes how this enables condition-based maintenance strategies, in which actions are triggered by measured equipment conditions rather than fixed maintenance intervals.

Research on industrial monitoring systems shows that continuous sensor feedback can detect deviations from normal operating conditions. In these systems, sensor data is transmitted to a centralised monitoring platform for automated analysis and fault identification [12]. Previous studies have explored both advanced data-driven anomaly detection methods and simpler condition-based approaches based on predefined thresholds and direct sensor measurements [13]. The literature also shows that these simpler approaches can provide sufficient reliability for industrial fault detection and operational monitoring.

### 1.5.3 Summary of Related Work

The reviewed literature shows that Industry 4.0 and Industrial IoT enable improved monitoring through interconnected devices, smart sensors, and real-time data processing. Previous studies highlight the importance of edge computing and embedded systems. They state that using this system can reduce latency by enabling local data analysis in an industrial environment. Another study also indicates that condition-based maintenance strategies can use sensor data and continuous monitoring to detect degradation and inform maintenance decisions before failure.

The literature also demonstrates the growing use of smart sensors and automated monitoring systems for industrial fault detection. Previous work has explored both advanced anomaly detection methods and simpler threshold-based approaches using direct sensor measurements. These studies show that reliable fault detection can be achieved through continuous sensor monitoring and comparison of expected system behaviour. The related work reviewed in this chapter influenced the design of the implemented system, particularly regarding local sensor data processing, condition-based monitoring, and anomaly detection in embedded industrial environments.

# Chapter 2

# Technical Background

## 2.1 Embedded Systems

Embedded systems are specialised computing systems that integrate hardware and software within a single product to perform dedicated functions. These systems are widely used across a broad range of applications, from consumer electronics such as interactive dolls [14] to highly complex systems such as robotics on spacecrafts [15]. A defining characteristic of embedded systems is the strong interdependence between their hardware and software components, commonly referred to as tight coupling [16]. This tight integration enables efficient, purpose-built operation but also imposes strict timing requirements.

Such systems are often subject to real-time constraints, requiring responses within specified time bounds. These constraints are typically expressed as deadlines, with system operations categorised as either feasible or late. A feasible operation is completed within or before its deadline, whereas a late operation occurs after the deadline and may degrade performance or cause system failure [17].

In recent years, the concept of edge processing has become increasingly relevant in embedded systems. Edge computing involves performing data processing closer to the data source, for example, the sensors, rather than relying on centralised cloud systems [18]. This approach reduces latency, decreases network bandwidth usage, and improves system reliability. This is increasingly important in industrial applications containing real-time constraints. By processing data locally, embedded systems can respond faster to events and continue operating even when network connectivity is unreliable.

## 2.2 Data Transmission with MQTT

The Internet of Things (IoT) refers to a network of physical devices embedded with sensors, software, and communication technologies that enable data exchange over the internet. Several communication protocols can be used to facilitate this exchange, including the Hypertext Transfer Protocol (HTTP), the Constrained Ap-

plication Protocol (CoAP), and the Message Queuing Telemetry Transport (MQTT) [19].

MQTT is a lightweight messaging protocol based on a publish/subscribe communication model, widely used in IoT applications due to its simplicity and low network overhead [20, 21]. The protocol operates on top of TCP/IP and is standardised by OASIS[22]. It is a Global non-profit organisation that works on developing and maintaining open standards, including MQTT. MQTT is designed for low impact, making it well-suited for smaller, resource-limited applications such as embedded systems and IoT applications.

In the publish/subscribe model, clients do not communicate directly with each other. Instead, they interact through a central component known as a broker. A client can act as a publisher or a subscriber. A publisher sends data to a broker while a subscriber receives data from the broker based on specified topics [19]. The broker manages message distribution, which includes ensuring that each client subscribed to that topic receives the message [21]. One commonly used broker is Eclipse Mosquitto, which was used in this project, a lightweight and open-source broker designed for efficient message handling across both low-power devices and more capable systems [23].

## 2.3 Smart Sensor and Condition-Based Maintenance

In industrial systems, sensors play a critical role in ensuring system functionality by providing different metrics, such as object position. Traditionally, sensors provide only binary outputs (on/off) depending on their state. This data is sufficient for simpler control logic, but binary signals are limited for condition monitoring, as gradual sensor performance degradation cannot be observed. For example, in a photoelectric sensor, the two states indicate only whether light is reflected to the sensor or whether an object is blocking it. Smart sensors extend sensor functionality by providing diagnostics and more in-depth data, along with binary signals. This data varies depending on the sensor type, but it may include signal strength, temperature and other diagnostics. These additional parameters enable monitoring of sensor condition and the detection of degradation patterns over time, thereby enabling the implementation of condition-based maintenance strategies.

Condition-based maintenance (CBM) is a widely used maintenance strategy in today's industry. It is based on continuous condition monitoring (CM) of measurable system parameters, obtained through sensor data and other diagnostic indicators. The objective of CBM is to perform real-time assessments of equipment health to identify signs of degradation used to support maintenance decision-making [24]. To detect degradation, CBM uses predetermined limits to indicate when a component's performance falls below an acceptable level [25]. These limits can be calculated by a single variable or a combination of variables and are often based on past data. When a component's performance falls below the predetermined limit, maintenance decisions are made.

For CBM to detect degradation, a single measurement that exceeds the predefined limit is insufficient. Sensor signals are often disturbed by noise, environmental changes, and normal signal fluctuations. This makes relying on a single measurement insufficient. Instead, CBM systems analyse measurements over time. Detecting anomalies becomes possible by evaluating patterns, such as a gradual decrease or increase in signal strength or an increase in measurement variation. Measurements over time also allow distinguishing between normal noise, system behaviour, and actual degradation [24, 25].

To enable the system to detect changes in signal strength despite noise and normal fluctuations, a standard incremental mean was used. This allowed the system to present long-term trends, such as signal strength degradation, while also reducing the impact of short-term variations such as noise.

To detect when the signal deviates significantly from its normal behaviour, the standard deviation was used [26, 27].

$$\mu_n = \frac{(n-1)\mu_{n-1} + x_n}{n} \tag{2.1}$$

Where:

- $\mu_n$ is the updated mean after $n$ observations,
- $\mu_{n-1}$ is the previous mean,
- $x_n$ is the newest observation,
- $n$ is the total number of observations.

The incremental mean ($\mu_n$) (Equation (2.1)) updates the average value each time a new measurement arrives. It does this by combining the previous mean ($\mu_{n-1}$) with the new value ($x_n$).

$$M_{2,n} = M_{2,n-1} + (x_n - \mu_{n-1})(x_n - \mu_n) \tag{2.2}$$

Where:

- $M_{2,n}$ is the running measure of variability after $n$ observations,
- $M_{2,n-1}$ is the previous measure of variability,
- $x_n$ is the newest observation,
- $\mu_n$ is the updated mean,
- $\mu_{n-1}$ is the previous mean.

Welford's equation (Equation (2.2)) provides a way to compute a running measurement of variability ($M_2$), which is used to calculate the real-time variance and standard deviation.

$$\sigma^2 = \frac{M_{2,n}}{n} \tag{2.3}$$

Where:

- $\sigma^2$ is the variance,
- $M_{2,n}$ is the running measure of variability after $n$ observations,

- $n$ is the total number of observations.

The variance (Equation (2.3)) represents the average squared deviation ($\sigma^2$) from the mean.

$$\sigma = \sqrt{\sigma^2} \tag{2.4}$$

Where:

- $\sigma$ is the standard deviation,
- $\sigma^2$ is the variance.

The standard deviation (Equation (2.4)) describes how much the measurements typically deviate from the mean. The standard deviation ($\sigma$) is obtained by taking the square root of the variance ($\sigma^2$).

Finally, anomaly detection is performed by checking whether a new value deviates from the mean by more than a chosen threshold, typically a multiple of the standard deviation (Equation (2.4)).

# Chapter 3

# Method

## 3.1 Development Process

The development of both the hardware and software components of the system was implemented using official documentation, manufacturing specifications and standard mathematical methods. The process began with the development of the overall system's hardware design, followed by the development of the communication method between components. Once communication was established, data processing and software development were carried out.

### 3.1.1 Hardware and System Design

The main components of the system were the IFM IO-Link master [28], the WTM1-2F SICK photoelectric proximity sensor [29], the WLG12P SICK photoelectric retro-reflective sensor [30], the IMC18 SICK inductive proximity sensor [31], and a Raspberry Pi 4 Model B [32]. The photoelectric sensors were used to detect the core's position in the machine. The inductive sensor was used to detect whether the machine was actively cutting by monitoring the movements of metal objects.

Because the IO-Link master was the central component of the system, the hardware and system design were built around it and the compatibility constraints it imposed. Communication between the Raspberry Pi and the IO-Link master was established via an Ethernet connection. To accommodate this connection, the Raspberry Pi 4 Model B was chosen. It had a necessary Gigabit Ethernet port, four USB ports, and two micro-HDMI ports. The USB ports enabled the Raspberry Pi configuration, and the micro-HDMI ports enabled a screen connection for the GUI to be displayed easily.

To establish the IO-Link master connection with the sensors, the ifm operating instructions were followed [33] together with ifm configuration software moneo [34]. Moneo was used to set the IO-Link master's IP address, and the IO-Link master's ifm-IoT Core Visualizer was then used, per the operating instructions, to configure the master and sensors.

### 3.1.2 Communication and Architecture

To establish communication between components, the MQTT protocol was selected based on its suitability for IoT applications [20, 21]. MQTT is a lightweight messaging protocol which follows the publish/subscribe architecture. This allows for decoupled communication between data producers and consumers. IoT systems commonly implement such an architecture due to its low network overhead and scalability.

The publish/subscribe model enabled multiple components in the system to access the same data stream without requiring direct connections, improving flexibility and modularity. To implement this method, Eclipse Mosquitto was used as the MQTT broker [23]. This was done due to its lightweight design and open-source availability, which made it suitable for resource-constrained embedded systems.

### 3.1.3 Data Processing and Analysis

The Raspberry Pi processed sensor data using statistical methods commonly used in predictive maintenance and condition-monitoring systems. The data collected from the broker contained an excessive number of unnecessary bits, which could be considered trash bits.

To correctly decipher the bitstream obtained from the broker, SICK official documentation was used for the WTM12F [29], WLG12P [30], and IMC18 [31] sensors. The bitstream was then shifted and edited based on which bits held the necessary values. The WTM12F was right-shifted by 2 bits to correct for the usable data starting on bit 2. A similar implementation was used on the IMC18, with the difference of right-shifting by 16 bits. The WLG12P operated differently, providing only a bit when it could see, changing between 0 and 1. To extract this bit, the AND operation with 1 was used to select only the least significant bit.

To calculate the average of incoming data, the standard cumulative approach to calculating the mean is not suitable in practice. This is because the mean and count can grow to infinity, potentially leading to instability and system crashes. To combat this, Welford's algorithm [27] was used. This is a numerically stable, single-pass algorithm that computes the mean and variance without storing all previous data points. This was chosen for its computational efficiency, with low memory usage and processing cost, making it a perfect candidate for a resource-constrained embedded system.

To detect anomalies, the 3-sigma rule was applied [26]. The rule assumes the values follow a normal distribution, which can be problematic in a noisy environment. Values affected by the noise may be detected as anomalies, increasing the false-alarm rate. To combat this, a limit was implemented so that if the program detected more than ten deviated values, it would alert. The number of deviated values was selected to balance the limit. Setting a limit that is too small can cause issues. The program would be prone to the same fault, and a too-large limit would mean that real anomalies could be missed.

With the assumption of normal distribution, approximately 99.73% of all values lie within three standard deviations from the mean. Values outside this range were therefore considered outliers and were flagged as anomalies, making this a practical method for monitoring system behaviour.

### 3.1.4 Software Engineering Practices

Version control was implemented using Git, an established software engineering practice for collaborative development and code revision [35]. The use of version control enables a team to track change history and revert to previous versions, and collaborate without overwriting work. The system was designed with a modular architecture, separating data acquisition, processing, and visualisation into independent components. This design approach is a common principle in software engineering for modularity and separation [36]. Such an approach improves maintainability and scalability and allows individual components to be developed and tested independently, making it suitable for programs in which several people work collaboratively.

## 3.2 Technical Design

The system was structured into two main components: hardware and software. Both components played a critical role in the system and together formed the complete embedded system.

The hardware component was responsible for data acquisition from the machine. Three SICK sensors were used: two photoelectric sensors (WTM12F and WLG12P) and one inductive sensor (IMC18). These sensors were connected to an IO-Link master using M12 cables. The IO-Link master was, in turn, connected to a Raspberry Pi via Ethernet.

To establish communication between the IO-Link master and the Raspberry Pi, the protocol MQTT was introduced. The IO-Link master was acting like the publisher. It sent the collected sensor data to the MQTT broker for the subscriber, which was the Raspberry Pi. The Raspberry Pi served as the subscriber receiving the data by subscribing to the topics.

The software component was responsible for processing and visualising the received data. Sensor data was first received through the MQTT communication layer and then passed to the processing module, where it was analysed before being forwarded to the user interface for visualisation.

Due to the sensors acting in such different ways, sensor-specific configurations were defined, allowing handling of the different sensor types without major code changes. These configurations contained parameters for determining differences between sensors.

The application maintained a limited history of recent sensor values for each sensor, allowing both real-time monitoring and short-term trend analysis. Incom-

ing data was continuously appended to this history and used for further processing.

The software tracked statistical properties for each sensor. These properties were average and standard deviation, which were used to detect anomalies by comparing incoming data against the standard deviation. When a deviation was detected, an alert was generated and presented to the user.

The system also included logic to detect inconsistent behaviour at the system level. For example, if one sensor remained inactive while the others were active, the system identified this as a potential fault condition and generated an alert.

To allow the user to view sensor data, a graphical user interface (GUI) was implemented. Here, the user could see the data (1), average (2), standard deviation (3), and status (4) for each sensor. A window containing alerts (5) was also available, along with a history button (6) for each sensor. The interface dynamically updated each data package received from the broker, providing continuous feedback to the user.




**Figure 3.1:** A picture showing the GUI implemented for the system. (1) Datastream, (2) average, (3) standard deviation, (4) sensor status, (5) alerts window, (6) history button

## 3.3 Testing

To verify the system's correctness and reliability, a structured testing approach was implemented. The testing process was divided into three levels:

- **Unit testing:** Individual software components were tested to ensure that each part of the code functions as intended.
- **Integration testing:** Communication between the Raspberry Pi and the I/O-Link Master was tested to ensure correct data transmission and interpretation.
- **System testing:** The complete system was evaluated in a controlled cutting machine environment under varying sensor conditions.

### 3.3.1 Unit testing

To verify that the software functioned as intended, a simulation program was developed to generate artificial data. This program created data packets resembling those transmitted by the MQTT broker. The generated packets were written to a JSONL file, which the software continuously read from. The updates were produced at the same interval as in the real system, specifically every 0.5 seconds. The simulated data initially represented normal operating conditions, with a 10% probability of gradual degradation in sensor values to mimic real-world wear and unpredictable behaviour.

### 3.3.2 Integration testing

To verify communication between the Raspberry Pi and the I/O-Link Master, the open-source MQTT Explorer was used. This tool allowed monitoring, interaction, and debugging of MQTT brokers. The connection was verified by subscribing to the broker's IP address, which was the local address (127.0.0.1) due to the direct Ethernet connection. The application provided real-time visualisation of transmitted data packets and their update frequency, confirming that an active connection was established. Additionally, the tool enabled inspection of the data payload, which was valuable for defining threshold values for idle and active sensor states and for designing realistic simulated test data.

### 3.3.3 System testing

After successful software and communication testing, the complete system was evaluated. The setup is shown in figure 3.2. The WLG12P (1) photoelectric sensor was installed on the left side of the saw blade (2) to detect whether a core was positioned for cutting. A second, more advanced photoelectric sensor, the WTM12F (3), was installed on the right side of the blade. This sensor performed a similar detection function but was positioned in an area more prone to dust accumulation to evaluate its robustness. An inductive sensor, IMC18 (4), was mounted to detect when a cutting operation was in progress. Instead of directly detecting the

rotation of the saw blade, the sensor was positioned farther back to detect the movement of an arm (5) that lowers the safety guard. The arm was chosen for its lower vibration compared to the saw blade, thereby reducing mechanical noise in the values. It is also a much safer installation and has less risk for the equipment during runtime.

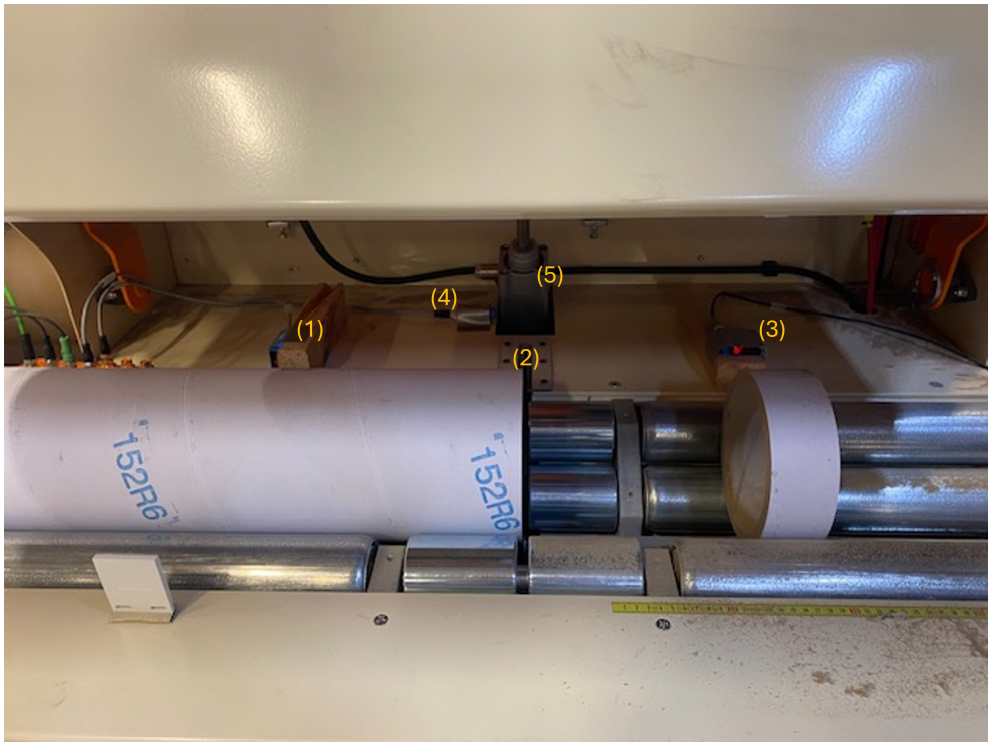



**Figure 3.2:** Figure of the system setup. (1) WLG12P photoelectric sensor, (2) saw blade, (3) WTM12F photoelectric sensor, (4) IMC18 inductive sensor, (5) safety guard arm.

### Standard Test

To enable the system to detect deviations and anomalies, the first test performed was a standard test. This test had no change in sensor positioning and was performed to collect baseline data that the system could use to evaluate future tests. To simulate a real cutting environment, the test began with a core pushed into the system from the left, as would occur in a standard larger system. This meant that the first photosensor, WLG12P, was blocked and entered the detecting state. Then, a small bit of an already cut core was placed in front of the WTM12F photosensor so that it would be in a detecting state as well. This was done due to the distance between the photosensors. If the tests had been performed with a single core that was detected by both sensors simultaneously, it would have wasted a lot of material. Once both cores were in place, a portion of the first core was cut,

triggering the induction sensor. After the cut, the cut portion was removed from the machine, the first core was pushed back out, and the second core was pushed out to simulate the entire core exiting the system. To maintain test consistency, this was the procedure for every cut in every test.

After the baseline values were established using the standard test, the different fault types defined in the requirements section were tested. To test all the different fault types, six different tests were developed. However, during each test, all the sensors were active and generating data, even if it wasn't the sensor being tested. This was done to collect as much data as possible from the sensors.

#### WTM12F Tests

For the smart photosensor, WTM12F, two different tests were performed. The first test simulated a scenario in which the sensor's environment generated significant dust and was performed to assess how the sensor's signal strength was affected by dust covering the lens. A small amount of dust was deliberately placed in front of the sensor between each of the ten cuts to simulate long use in a dusty environment. The second test for the smart sensor simulated the sensor vibrating out of position and was performed to assess how its signal strength was affected. Between each of the ten cuts, the sensor was rotated by approximately the same amount. Both tests were performed because the testing environment and the problem it simulated are real factors affecting sensors in a real system.

#### IMC18 Tests

The inductive sensor, IMC18, was also tested with two different tests. The first test simulated the sensor vibrating out of position. It was performed to assess its ability to withstand such a condition. Ten cuts were performed, and between cuts the sensor was rotated by approximately the same amount to simulate it vibrating out of position. The second test was performed to determine whether the sensor was affected by a physical object blocking its detection of the metal, and if so, how much the object affected the sensor signal. Due to a dust extraction system located near the sensor, dust could not be used. Instead of achieving this, tape was used, with a layer being added between the cuts.

#### WLG12P Tests

Two different test scenarios were performed for the WLG12P photosensor and its reflex. Because the sensor is highly dependent on the reflex, even small changes can cause it to stop working. The first test assessed its sensitivity to vibrations by rotating the reflex by a small amount between each cut. Because the initial setup had the light from the sensor reflected in the middle of the reflex, the reflex was rotated only to the right. But to test whether the angle of no reflections was the same on both sides, the reflector was rotated to the left until the sensor received no light. Both the angles of no received light to the right and left were measured.

The second test for the sensor involved simulating dirt on the reflector. To achieve this, dust was deliberately placed on the reflector between the cuts until the sensor stopped receiving light.

# Chapter 4

# Implementation

## 4.1 System Setup

The system was assembled using three SICK IO-Link-compatible sensors: a WTM1-2F photoelectric proximity sensor, a WLG12P retro-reflective sensor, and an IMC18 inductive proximity sensor. All sensors were connected to the IFM AL1352 IO-Link master via standard M12 connector cables.

The IO-Link master was then connected to the Raspberry Pi 4 Model B using an M12-to-Ethernet cable. The Raspberry Pi used the Linux-based Raspberry Pi OS and was the processing unit for all the data. A display was connected via one of the micro-HDMI ports to enable visualisation of the GUI and the system output. A keyboard and mouse were connected to two USB ports for system configuration and interaction.

## 4.2 IO-Link Configuration

The IO-Link master and the sensors were configured using IFM configuration software moneo and the IO-Link master's ifm-IoT Core Visualizer [33, 34]. Moneo was used to set the IO-Link master's IP address to 169.254.1.1. The IP address was then used to call: http://169.254.1.1/web/subscribe to reach the ifm-IoT Core Visualizer. The sensors and IO-Link master were then configured by importing the IODD (IO Device Description) files for each sensor, enabling communication between the sensors and the master. The IoT Core Visualizer was also used to configure the IO-Link master transmission rate to 500 ms.

## 4.3 Communication

To enable MQTT communication between the IO-Link master and the Raspberry Pi, Eclipse Mosquitto was implemented as the broker, running locally on the Raspberry Pi. The data from each sensor was published to its dedicated MQTT topic. The system then used a publish/subscribe architecture to receive sensor data in

real time. The system used events: incoming messages triggered data processing functions, eliminating the need for polling. To receive the data in a human-readable state and to save it for later use, the following code line was run in the terminal:

**Listing 4.1:** Terminal command for data storage

```
mosquitto_sub -h [IP-adress] -t [Broker-topic] >> [Filename].jsonl
```

Upon receiving a message from the broker, it stores it in a file using the JSON Lines (JSONL) format. The JSONL format means that each message was written as a separate JSON object on a new line. This enables efficient data writing without the need to rewrite the entire file [37]. This approach simplified data logging and enabled the system to handle dynamically updating data streams in a lightweight manner.

This approach has limitations when scaling into larger data volumes. When the file grows, operations such as reading and searching become less efficient. This makes it a vulnerability for a long-term storage solution. Solutions such as a database or a streaming platform will be better suited for such a system. While the current implementation was sufficient for the scope of this project, an alternative long-term storage solution should be considered.

## 4.4 Data Processing

To receive data from the JSONL file, a separate module (Sensor.py) was implemented. This module was invoked from the main application through the `start_reading` method. The file was opened and continuously monitored using the Python JSON library. A file-following mechanism was implemented to process new data as it was appended to the file. The system continuously reads the new lines from the file. If no new data was available, the process paused briefly before retrying.

Each line was parsed and forwarded to a processing function. The sensor address and payload were extracted; the address identified the sensor source, and the payload contained the measured data.

Since the payload contained multiple data fields, a filtering step was required to extract the relevant sensor value.

This step ensured that only the relevant IO-Link process data was selected. After validating the data and converting it to an integer, the processed value was forwarded to the graphical interface via a callback.

The sensor data received by the Raspberry Pi consisted of raw bitstreams of varying lengths. To extract the meaningful bits, bitwise operations were implemented in the software based on each sensor's data format.

For the WTM12F photoelectric smart sensor, the current receiver level was extracted by right-shifting the raw data value by 2 bits. Thereby ignoring bits 0 and 1, which were irrelevant to the current receiver level, according to its data sheet. The input value represents the raw data received from the IO-Link master. The raw data value gets shifted two bits to the right.

For the IMC18 inductive sensor, the raw data bitstream was 32 bits, where only bits 16-31 were relevant. These bits were extracted by right-shifting the raw data value by 16 bits. The input value represents the raw data received from the IO-Link master. The raw data value gets shifted 16 bits to the right.

For the WLG12P photoelectric sensor, only the least significant bit indicates if the sensor detects an object. To get the correct bits, the AND operation was used to evaluate the least significant bit of the raw data value. A value of 1 indicates detection, while 0 indicates no detection. To easily represent this in the UI, the scaling function was implemented to show a value of 100 counts if detection and 0 during no detection.

## 4.5 Software Implementation

The software was implemented in Python using a modular, event-driven architecture. These modules were divided into data collection, processing, GUI, and history.

A dedicated module (Sensor.py) was implemented to read and parse data from the JSONL file. Another module (SensorGUI.py) is responsible for presenting the data and alerts to the user. Data processing occurs in one module (Processing.py), and the history is displayed in another (history_window.py).

The data flowed through the different files, where it was read live from the JSONL file by the Sensor module, which parsed, filtered, and processed it into actual values before forwarding it to the SensorGUI. In this module, the raw data is shown to the user as well as the results after further processing, which happens in the Processing module. Here, the average, standard deviation, and alerts are checked, then sent back to the SensorGUI for presentation yet again. history_window is a separate module that, when called, reads the entire JSONL file and presents the raw history data and the average to the user in a separate window.

These modules follow an event-driven architecture in which incoming data triggers changes and methods within the system. Even the history_window depends on a button event.

To implement the modules, standard Python libraries such as json, threading, and collections were used to support data parsing and simplify programming.

## 4.6 User Interface

The graphical user interface was implemented using the Python library Tkinter, which enabled the visualisation of sensor data in real time. The GUI displayed the live sensor data and served as the main interface for monitoring the system's behaviour.

Each sensor had a third of the main window dedicated to it, with a real-time graph showing the current value, the standard deviation thresholds, and a label

showing the running average together with the sensor's operational status. The operational status indicated whether the sensor was active or idle, with thresholds set based on initial and base tests. The real-time graph used a fixed-size buffer set to 100 data points. This buffer enabled continuous graph updates without excessive memory usage. The main UI window also included buttons for each sensor's data history. When pressed, the buttons opened a new window showing a graph of that sensor's historical data along with the running average since the program was initialised. This allowed for observations of how the sensor had performed over time.

# Chapter 5

# Results

## 5.1 Graphical User Interface

Figure 5.1 shows the graphical user interface during regular system operation, free of any introduced faults. The Figure presents active data from three sensors during several process cuts. The raw signal is shown as a blue line, the mean value as an orange-dashed line, and the standard deviation as two red-dotted lines.

During the cutting process, variations in sensor values can be observed, while the signals return to a stable level when the system is idle. The numerical values for each sensor, along with the corresponding average, are displayed in the interface.

In addition to the sensor data, a system log window is shown. Here, generated alerts and warnings are shown, including when a sensor value deviates significantly from the expected range. Figure 5.2 illustrates this behaviour, where the IMC18 sensor produces values outside the standard deviation limits, resulting in multiple alerts. Each alert includes a timestamp, sensor identifier, and deviation magnitude.

A separate history view is shown in Figure 5.3. This view displays previously recorded sensor data over time, along with their corresponding averages. It can be observed that the average adapts to changes in sensor values and stabilises during periods of consistent measurements. A gradual stabilisation of the average values can be observed as more data points are collected.

## 5.2 Tests Under Normal Conditions

The Standard tests were performed to establish the system's baseline behaviour and set baseline parameters for the different test scenarios. The values from these tests are shown in Figures 5.4, 5.5, 5.6. Two different standard tests were performed on two different testing days. In the graphs, there is a gap at approximately 450 on the x-axis, which represents the two different tests.

Figure 5.4 shows the WTM12F photoelectric sensor values. When at 16000, it is in the detective state, which indicates a core entering the system, and when

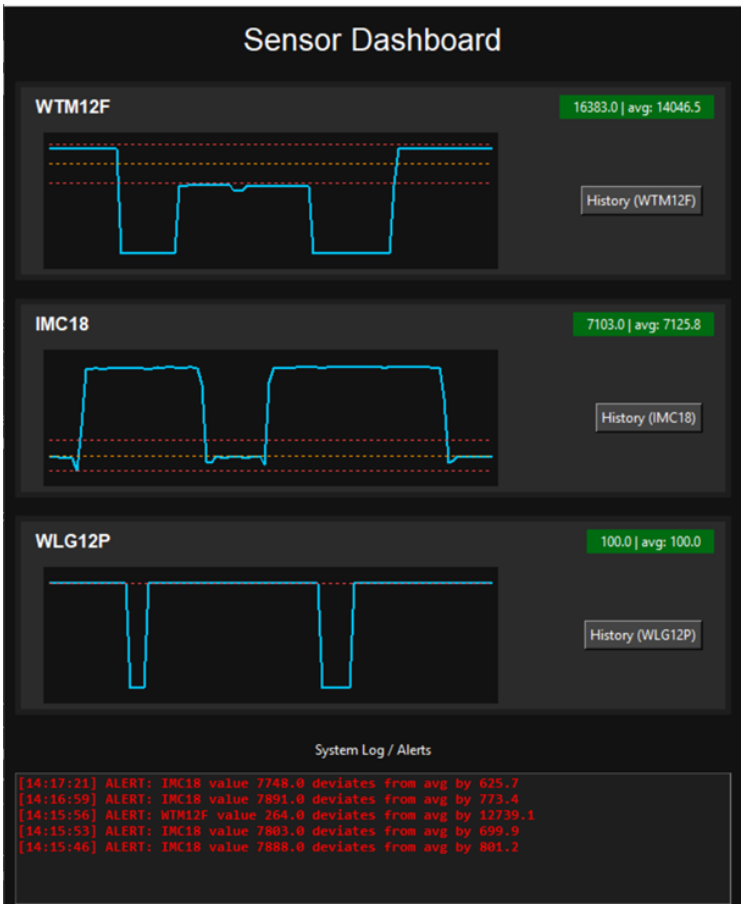



**Figure 5.1:** The system operating under normal conditions, where each cut remains within the standard deviation limits, and no errors have been detected since system startup.

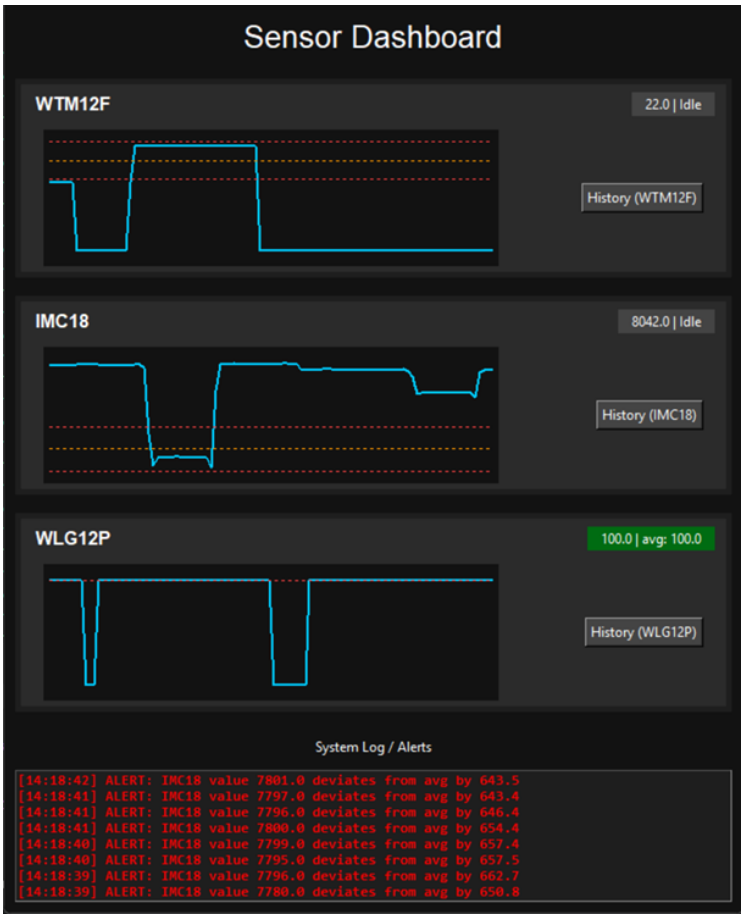



**Figure 5.2:** The system is operating under abnormal conditions, with the IMC18 sensor rotated out of position, causing measured values to exceed the standard deviation limits and triggering multiple alerts.

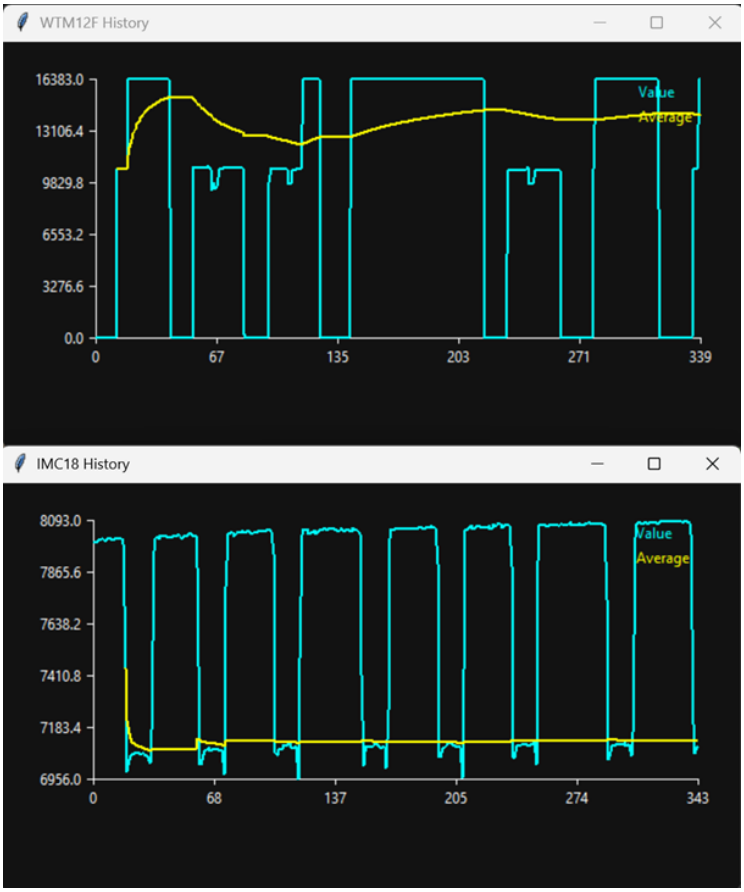



**Figure 5.3:** Historical view of the WTM12F and IMC18 sensor values during normal operation. The blue line represents the raw sensor values, while the yellow line represents the calculated average.

at 0, it indicates that the sensor is idle. During the second half of the graph, i.e., the second standard test, there are deviations in which the current receiver level during detection only reaches approximately 10700.

In Figure 5.5, the WLG12P proximity photoelectric sensors are in two states: actively detecting (1) and idle (0). The detecting state represents when a core is actively in the system, and the idle state indicates when a core is not in the system.

For the inductive sensor in Figure 5.6, the signal strength starts at approximately 8000 when idle. The sensor was located to detect when the cuts were made, so the dips in the values indicate when they were made. Both the first and second standard tests showed an increase in value over time. The metrics used to describe the deviations below represent the mean difference between the idle baseline and the lowest point of the active detection dips. The first test, in the first part of the graph, showed a difference of approximately 200 between the idle and active states. In contrast, the second test in the second part of the graph had a mean difference of approximately 1000, with a bigger variance in the dip-values.

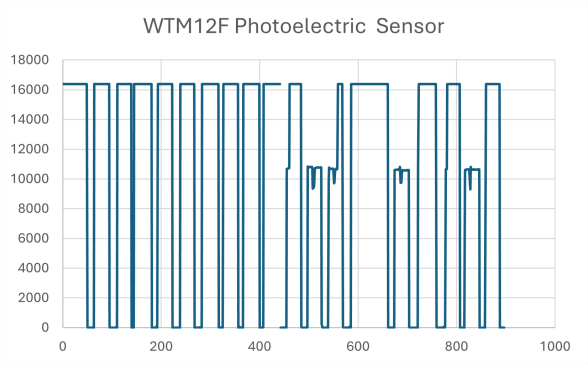



**Figure 5.4:** Graph showing WTM12F sensor values during normal operating conditions.

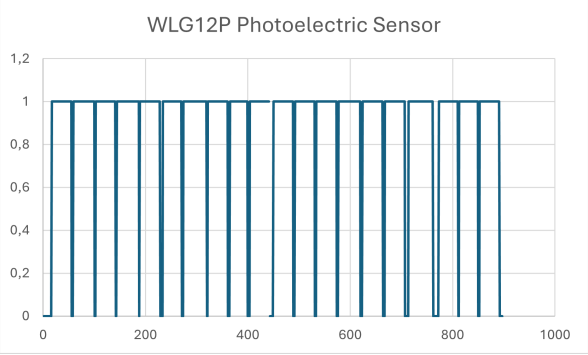



**Figure 5.5:** Graph showing WLG12P sensor values during normal operating conditions.

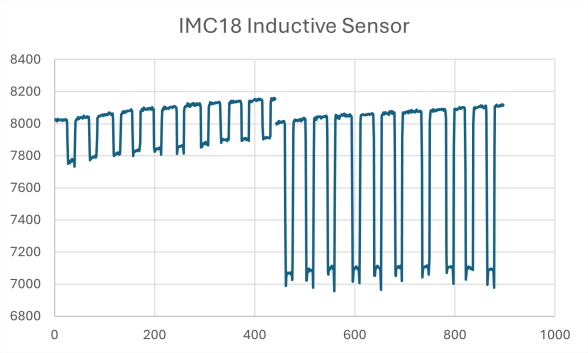



**Figure 5.6:** Graph showing IMC18 sensor values during normal operating conditions.

## 5.3 IMC18 Tests

The rotation test of the IMC18 inductive sensor was performed to simulate the sensor vibrating out of position. Between each cut, the sensor was rotated slightly to the right, away from the cylinder arm. Figure 5.7 shows the 5 cuts, with the signal dip becoming smaller for the first two cuts and then becoming unclear as the cuts progressed. Since the value did not fall below the idle threshold for the last three cuts, the test duration was shortened to give more time for other tests. The first signal dip matches the first standard test because they were conducted on the same day. During the test, it was observed that small changes in the sensor position produced noticeable changes in the measured signal.

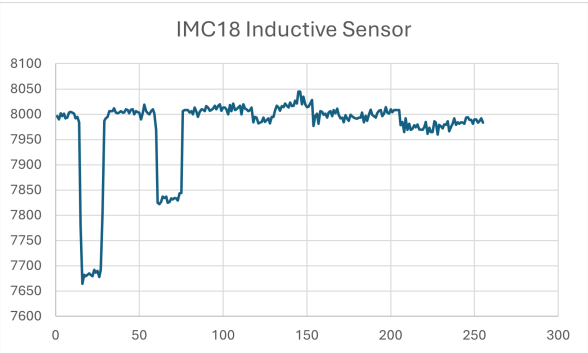



**Figure 5.7:** Graph showing IMC18 sensor values during the rotation test used to simulate vibration-induced sensor misalignment.

To analyse whether physically blocking the IMC18 inductive sensor affected it, a test was conducted in which increasing numbers of layers of tape were placed in front of the sensor between each cut. As shown in Figure 5.8, there were some variations between the first cut and the following cuts, but the signal strength did not change noticeably.

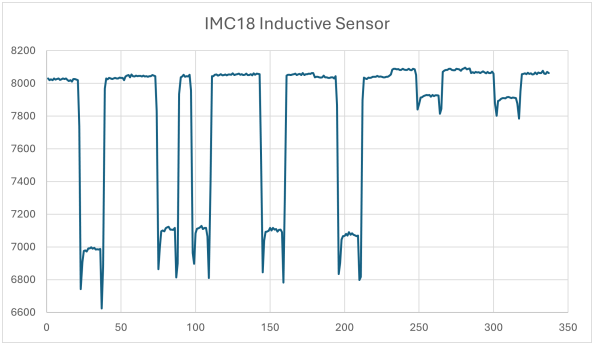



**Figure 5.8:** Graph showing IMC18 sensor values during the tape test, which simulates dust obstructing the sensor surface.

## 5.4 WTM12F Tests

The rotation test of the WTM12F photoelectric sensor simulated the sensor vibrating out of position. During the test, 10 cuts were made, and the sensor was rotated between each cut, as shown in Figure 5.9. To simulate a real environment, a bit of the core was placed in front of the sensor at the start of each cut and removed afterwards to simulate the core leaving the system. As shown in Figure 5.9, the value ranges from approximately 16300 to 0 depending on the active states. During the fourth cut, an anomaly is visible in which the value spiked and then dropped to around 5400. After seven cuts, the signal strength dropped noticeably to 6000 and continued to decline in subsequent cuts. It was observed that the current receiver level (signal strength) began to drop when the sensor was rotated by a large angle, as shown in Figure 5.9.

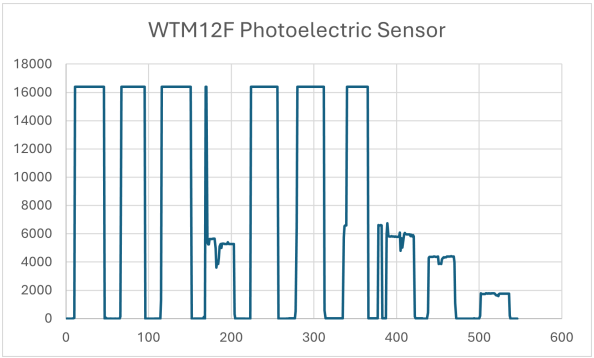



**Figure 5.9:** Graph showing WTM12F sensor values during the rotation test used to simulate vibration-induced sensor misalignment.

To simulate an environment where a lot of dust is generated and accumulates over time in front of the sensor, a test was conducted to analyse how the signal strength is affected. In this test, increasing amounts of dust were deliberately placed in front of the WTM12F photoelectric sensor between cuts. As shown

in Figure 5.10, the current receiver level live value decreased as more dust was placed in front of the sensor.

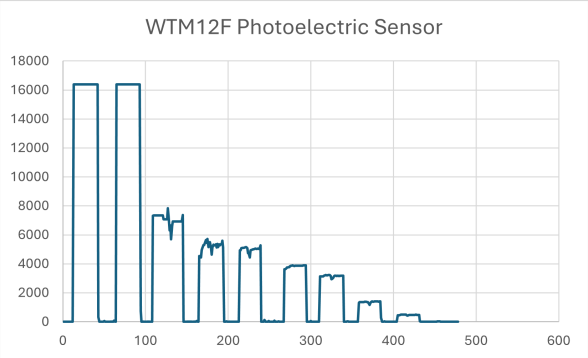



**Figure 5.10:** Graph showing WTM12F sensor values during the dust test, which simulated dust accumulation on the sensor surface.

## 5.5 WLG12P Tests

To simulate how vibrations affected the WLG12P photoelectric proximity sensor, a rotation test of the sensor's reflex was performed. A total of 5 cuts were made, and in between each cut, the reflexes were slightly rotated to the left. Because this photosensor needed reflected light to work, the exact angle at which it stopped receiving it was measured. When rotated to the left, the sensor stopped receiving reflected light at 17.7 degrees, and when rotated to the right, it stopped receiving reflected light at 21.6 degrees.

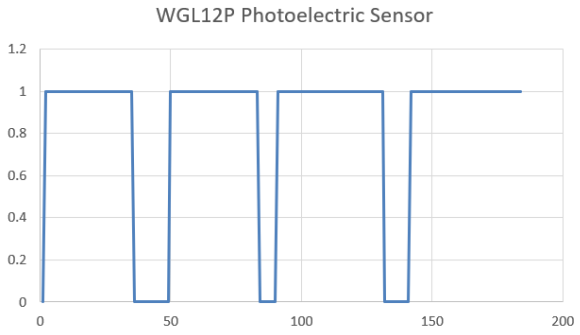



**Figure 5.11:** Graph showing WLG12P sensor values during the rotation test used to simulate vibration-induced sensor misalignment.

# Chapter 6

# Discussion

## 6.1 Software Performance and Requirement Verification

The results of the software implementation indicated that the system met the defined software requirements. The system displayed live values, provided a historical view of sensor values, including a running average, and analysed the entered values in real time. Deviations in the values were also detected using the running average and standard deviation, and alerts were consistently triggered for sudden changes in the signal.

## 6.2 Hardware Mounting Limitations and Baseline Variability

Because the setup inside the cutting machine was temporary and the mounting options for the sensors were limited, this could have affected the test results. Each simulation and test required the sensors to be moved or tampered with to achieve the required environment. Because the sensors lacked a stationary reference point to reset to, it was very difficult to consistently reset the system between tests. As shown in the rotation test results for the IMC18 inductive sensor, small rotations of the sensor significantly affected the signal value. This meant that even minor inconsistencies in the setup could have affected the results. An example of this is the two standard tests, where the IMC18 sensor values differed by approximately 1000 counts. This difference could have resulted from the sensor not being in the same position.

To address potential sensor displacement in the system, the active and idle states were used. Instead of calculating an average that included each sensor value, the running average included only values obtained while the system was active. The thresholds for the idle states were set based on data from the standard tests. This allowed the running average to better reflect the system's true average, since the idle values were vastly different from the active ones.

Three main limitations affected the evaluation. Firstly, the anomaly detection

relies on a threshold of three standard deviations from the running average. These values are calculated solely from values recorded during its active state, enabling active-state filtering and improving the accuracy of the average. Each sensor maintains its own running average and standard deviation. The 3x multiplier remains the same across all sensors. This means that a sensor with a naturally high noise floor will produce a large standard deviation, pushing the fault threshold far from the mean and potentially hiding real faults.

Second, the number of cutting cycles recorded per test was small, limiting the statistical confidence of the results. A larger dataset of cuts would be needed to establish normal operating variation and validate the reliability of the chosen thresholds.

Third, the lack of a fixed sensor mounting reference meant that the physical setup on the machine could not be the same between testing days. As seen in the IMC18 results, small differences in sensor position resulted in significant variations in the values. This makes it difficult to determine whether observed differences in testing reflect genuine changes in machine condition or are simply inconsistencies in the experiment setup.

## 6.3 Analysis of Sensor Anomalies and Environmental Disturbances

When the first standard test, shown in the first half of Figure 5.6, was performed, an incline in the values during the test was observed. A possible explanation for this inclination is that the sensor vibrated out of position due to the mounting situation. Upon investigation, the sensor was found to be slightly loose, suggesting it may have vibrated out of position. This can be supported by the results of the IMC18 rotation test, because of how small changes can have a significant impact on the system. This also shows the problem and fault type of vibration in such systems.

In the second standard test, some anomalies appear in the second part of Figure 5.4 for the WTM12F photoelectric sensor. This anomaly was also observed during the rotation test of the WTM12F sensor and during some tests of other sensors. When investigated, the source of the anomaly was believed to be the coloured letters on the sides of a core. After further investigation, it was found that when the WTM12F light reflected onto the coloured letter. The live values matched those shown in the figure, proving the theory.

During testing, irregular behaviour was observed in the IMC18 sensor when external factors such as dust and debris were introduced. Inductive sensors operate by generating an electromagnetic field and detecting changes in this field caused by nearby metallic objects. Since materials such as dust or non-metallic tape are not expected to significantly affect this field, the sensor output should remain unaffected. The result shown above indicates that a minor deviation occurs after the application of the first tape and remains consistent for the next four

measures. After the fifth tape was applied, a noticeable anomaly occurred. This was unexpected given the tape's nonmagnetic properties, which should not have affected the values in this way. The most likely explanation is that the inductive sensor, which is sensitive to small variations, was moved during the application of the fifth tape layer.

The test results where dust was placed in front of the WTM12F sensor show a clear decrease in the measured signal level. This behaviour is expected, as increasing dust levels reduce the amount of reflected light reaching the receiver. It can be observed that even a small amount of added dust significantly reduces the sensor value. This fall does not provide a fault reading from the system, as it is still higher than the reading when the sensor falls on the black text on the cores. However, continued exposure to this dusty environment will lower the average. It can be observed that increasing dust levels further degrade the data, allowing the system to detect an anomaly.

## 6.4 Societal, Ethical, and Privacy Requirements

When implementing an industrial monitoring system within the Industry 4.0 framework, it is important to evaluate its impact beyond technical performance. This section discusses the prototype's implications regarding economic factors, environmental sustainability, workplace safety, and data privacy.

### 6.4.1 Economic Requirements

The system presented in this report offers a cost-effective approach to sensor condition monitoring. By using relatively low-cost components, such as Raspberry Pi and IO-Link sensors, companies can reduce the cost of implementing condition-based monitoring. The system analyses sensor data degradation over time and alerts for possible problems, enabling predictive maintenance planning. This can reduce unexpected operational downtime by optimising service intervals and preventing premature, costly hardware replacements, improving cost efficiency.

### 6.4.2 Environmental Sustainability

The environmental footprint for the embedded system could be argued in two ways. One way is to see it as introducing additional components to the system. These microcontrollers and sensors all contribute to electronic waste and consume additional electrical power. On the other hand, the primary goal of a condition-based monitoring system is to identify inefficiencies and sensor faults before they cause sustainable mechanical damage or material waste. It does this by ensuring that the cutting machine operates under optimal conditions, thus minimising resource consumption and extending the machine's lifespan.

### 6.4.3 Workplace Safety

Industrial environments present physical risks to machine operators. The system directly improves workspace safety by monitoring critical mechanical components. For example, the IMC18 monitors the safety guard arm that protects the user from the saw blade during operation. This allows the system to flag dangerous deviations in machine behaviour immediately. In this example, when the safeguard fails to lower. This quick alert generation serves as an extra layer of diagnostic security, preventing accidents and reducing the risk of injury during machine operations.

### 6.4.4 Integrity and GDPR Compliance

Data privacy and regulatory compliance, such as the General Data Protection Regulation (GDPR) [38], are crucial when implementing monitoring software. The system's key features include its complete standalone functionality and non-cloud connectivity, as well as its use of edge computing, all of which enhance data privacy. The system exclusively logs raw data from physical sensors, including signal strengths, averages, and binary states. These data points consist solely of technical telemetry and are not associated with any personal information. Consequently, the system does not store or collect personal data, including employee identities, facial images, or user tracking metrics. As a result, the system poses no privacy risks and fully complies with the GDPR's design mandates [38].

# Chapter 7

# Conclusion

This project demonstrated a working prototype of a real-time embedded monitoring system for industrial sensors, with basic anomaly detection and deviation detection for sensor values. The system was based on IO-Link communication, MQTT, and a Raspberry Pi. This system successfully implemented continuous sensor data collection, processing, and visualisation. This implementation also addressed the initial challenge of limited available sensor data by baseline measurements through controlled experiments, which serve as a basis for detecting deviations without relying on historical datasets.

The results show that the statistical methods used, running averages and standard deviation, can be used to identify deviations in sensor values. However, the methods can still be affected by environmental noise, which could reduce their effectiveness in a more complex industrial environment.

Overall, the project showed the feasibility of a non-cloud-connected monitoring system for industrial applications. The developed prototype fulfilled the functional and non-functional requirements by using real-time sensor monitoring and anomaly detection under controlled conditions. The project was affected by delays in obtaining hardware components, reducing the time available for software refinement. Despite this, a functional prototype was successfully implemented and tested, which met the defined success criteria by generating alerts for introduced fault types and responding within the required sampling interval of 0.5 seconds. The results also show clear possibilities for future development and further industrial validation.

## 7.1 Future Work

While the prototype meets the project's requirements, future work will focus on three concrete items to enhance performance, scalability, and validation:

- Data Storage Infrastructure: Transition from the current JSONL file-based logging, which creates a scalability bottleneck, to a dedicated database or a streaming platform. This will eliminate efficiency issues during reading

and searching, enabling robust long-term storage, querying, and analysis of large industrial data volumes.

- Predictive Learning Models: Advance the system from the current statistical anomaly detection (rolling averages and standard deviations) to predictive maintenance. This will involve implementing machine learning model families (such as Random Forests or Neural Networks) trained on larger historical datasets to predict equipment and sensor failures before they occur.
- Industrial Validation and Test Scaling: To transition the system from a manual testing environment to a fully automated production line, future work will focus on scaling test coverage. The monitoring software should be validated through continuous operations, with more than 1000 automated cuts, to build a larger dataset. This extensive testing will evaluate the statistical algorithms against a higher volume of fault occurrences to ensure high alert precision and eliminate false positives.

# Bibliography


[1] S. Rane, P. Shah, and R. Sekhar, "Survey of technologies for industry 4.0," in *2022 6th International Conference On Computing, Communication, Control And Automation (ICCUBEA*, 2022, pp. 1–6. DOI: 10.1109/ICCUBEA54992.2022.10010837.

[2] D. Shylla, P. Shah, R. Sekhar, and M. D, "Embedded systems in industrial automation 4.0," in *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2023, pp. 1–6. DOI: 10.1109/ICCCNT56998.2023.10307700.

[3] M. Compare, P. Baraldi, and E. Zio, "Challenges to iot-enabled predictive maintenance for industry 4.0," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 4585–4597, 2020. DOI: 10.1109/JIOT.2019.2957029.

[4] M. Achouch, M. Dimitrova, K. Ziane, S. Sattarpanah Karganroudi, R. Dhouib, H. Ibrahim, and M. Adda, "On predictive maintenance in industry 4.0: Overview, models, and challenges," *Applied Sciences*, vol. 12, no. 16, 2022, ISSN: 2076-3417. DOI: 10.3390/app12168081. [Online]. Available: https://www.mdpi.com/2076-3417/12/16/8081.

[5] *Iec 61131-9: Programmable controllers – part 9: Single-drop digital communication interface for small sensors and actuators (io-link)*, IO-Link communication standard, International Electrotechnical Commission, 2013.

[6] M. Soori, B. Arezoo, and R. Dastres, "Internet of things for smart factories in industry 4.0, a review," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 192–204, 2023, ISSN: 2667-3452. DOI: https://doi.org/10.1016/j.iotcps.2023.04.006. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667345223000275.

[7] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in industrial internet of things: Architecture, advances and challenges," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2462–2488, 2020. DOI: 10.1109/COMST.2020.3009103.

[8] A. Cachada, J. Barbosa, P. Leitão, C. A. Geraldes, L. Deusdado, J. Costa, C. Teixeira, J. Teixeira, A. H. Moreira, P. M. Moreira, and L. Romero, "Maintenance 4.0: Intelligent and predictive maintenance system architecture," in *2018 IEEE 23rd International Conference on Emerging Technologies and Fac-*

*tory Automation (ETFA)*, vol. 1, 2018, pp. 139–146. DOI: 10.1109/ETFA.2018.8502489.

[9] MIMOSA, *Mimosa osa-cbm: Open system architecture for condition-based maintenance*, Accessed: 2026-05-09. Archived version used: https://web.archive.org/web/20260124061340/https://www.mimosa.org/mimosa-osa-cbm/, 2010. [Online]. Available: https://www.mimosa.org/mimosa-osa-cbm/.

[10] S. M. Ekene, C. N. Micheal, C. O. Kennedy, and E. C. Genevra, "Self diagnostic system for predictive maintenance of traffic light control system," in *2013 IEEE International Conference on Emerging & Sustainable Technologies for Power & ICT in a Developing Society (NIGERCON)*, 2013, pp. 308–313. DOI: 10.1109/NIGERCON.2013.6715669.

[11] M. Pech, J. Vrchota, and J. Bednář, "Predictive maintenance and intelligent sensors in smart factory: Review," *Sensors*, vol. 21, no. 4, 2021, ISSN: 1424-8220. DOI: 10.3390/s21041470. [Online]. Available: https://www.mdpi.com/1424-8220/21/4/1470.

[12] A. Jaramillo-Alcazar, J. Govea, and W. Villegas-Ch, "Anomaly detection in a smart industrial machinery plant using iot and machine learning," *Sensors*, vol. 23, no. 19, 2023, ISSN: 1424-8220. DOI: 10.3390/s23198286. [Online]. Available: https://www.mdpi.com/1424-8220/23/19/8286.

[13] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, "Machine learning for anomaly detection: A systematic review," *IEEE Access*, vol. 9, pp. 78 658–78 700, 2021. DOI: 10.1109/ACCESS.2021.3083060.

[14] VTech, *Baby amaze™ peek & learn doll™*, Accessed: 2026-05-08. Wayback Machine snapshots exist but are not fully renderable; an archived copy is included in Appendix A., 2015. [Online]. Available: https://www.vtechkids.com/product/detail/16632.

[15] NASA Johnson Space Center, *Robotics*, Accessed: 2026-05-08. Archived version used: https://web.archive.org/web/20260330080837/https://www.nasa.gov/reference/jsc-robotics/, 2026. [Online]. Available: https://www.nasa.gov/reference/jsc-robotics/.

[16] M. Grimheden and M. Törngren, "What is embedded systems and how should it be taught?—results from a didactic analysis," *ACM Trans. Embed. Comput. Syst.*, vol. 4, no. 3, pp. 633–651, 2005, ISSN: 1539-9087. DOI: 10.1145/1086519.1086528. [Online]. Available: https://doi.org/10.1145/1086519.1086528.

[17] J. R. Haritsa, M. J. Carey, and M. Livny, "On being optimistic about real-time constraints," in *Proceedings of the Ninth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, ser. PODS '90, Nashville, Tennessee, USA: Association for Computing Machinery, 1990, pp. 331–343, ISBN: 0897913523. DOI: 10.1145/298514.298585. [Online]. Available: https://doi.org/10.1145/298514.298585.

[18] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198.

[19] R. N. Boublenza, B. Benadda, and A. R. Borsali, "Hardware design of a low-cost iot current monitoring with mosquitto mqtt broker," in *2024 2nd International Conference on Electrical Engineering and Automatic Control (ICEEAC)*, 2024, pp. 1–3. DOI: 10.1109/ICEEAC61226.2024.10576213.

[20] S. K. Chakravarty, A. Batra, N. Singh, and G. Kumar, "The effect of tls encryption on mqtt protocol security and performance in meteorological-iot networks," in *2025 IEEE International Conference on Computer, Electronics, Electrical Engineering & their Applications (IC2E3)*, 2025, pp. 1–5. DOI: 10.1109/IC2E365635.2025.11167073.

[21] A. P. S and S. M. D. Kumar, "Delay estimation of healthcare applications based on mqtt protocol: A node-red implementation," in *2022 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, 2022, pp. 1–6. DOI: 10.1109/CONECCT55679.2022.9865759.

[22] OASIS Open. "Oasis open." Accessed: 2026-05-08. Archived version used: https://web.archive.org/web/20260603085830/https://www.oasis-open.org/. [Online]. Available: https://www.oasis-open.org/.

[23] R. A. Light, "Mosquitto: Server and client implementation of the mqtt protocol," *Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017. DOI: 10.21105/joss.00265. [Online]. Available: https://doi.org/10.21105/joss.00265.

[24] R. Ahmad and S. Kamaruddin, "An overview of time-based and condition-based maintenance in industrial application," *Computers & Industrial Engineering*, vol. 63, no. 1, pp. 135–149, 2012, ISSN: 0360-8352. DOI: https://doi.org/10.1016/j.cie.2012.02.002. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360835212000484.

[25] H. N. Teixeira, I. Lopes, and A. C. Braga, "Condition-based maintenance implementation: A literature review," *Procedia Manufacturing*, vol. 51, pp. 228–235, 2020, 30th International Conference on Flexible Automation and Intelligent Manufacturing (FAIM2021), ISSN: 2351-9789. DOI: https://doi.org/10.1016/j.promfg.2020.10.033. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2351978920318886.

[26] D. Montgomery, *Introduction to Statistical Quality Control* (EMEA edition), Eighth. Wiley, 2019, ISBN: 9781119657118. [Online]. Available: https://www.wiley.com/en-br/Introduction+to+Statistical+Quality+Control%2C+8th+Edition-p-9781119399308.

[27] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962. DOI: 10.1080/00401706.1962.10490022. eprint: https://www.tandfonline.com/doi/pdf/10.1080/00401706.1962.10490022. [Online]. Available: https://www.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022.

[28] IFM Electronic, *Al1352 io-link master datasheet*, Accessed: 2026-02-27, 2023. [Online]. Available: https://www.quicktimeonline.com/assets/images/pdf/IFM%20Electronc/datasheet-AL1352.pdf.

[29] SICK AG, *Wtm12f-24161220a00zdzzzzzzzzz1*, Datasheet for IO-Link WTM12F photoelectric sensor, SICK AG, 2026. [Online]. Available: https://www.sick.com/media/pdf/5/55/055/dataSheet_WTM12F-24161220A00ZDZZZZZZZZZ1_1152326_en.pdf.

[30] SICK AG, *Wlg12p-24162120a00zdzzzzzzzzz1*, Datasheet for IO-Link WLG12P photoelectric sensor, SICK AG, 2026. [Online]. Available: https://www.sick.com/media/pdf/8/88/188/dataSheet_WLG12P-24162120A00ZDZZZZZZZZZ1_1146667_en.pdf.

[31] SICK AG, *Imc18-12nppvc0sa00*, Datasheet for IO-Link IMC18 inductive sensor, SICK AG, 2026. [Online]. Available: https://www.sick.com/media/pdf/8/58/958/dataSheet_IMC18-12NPPVC0SA00_1079295_en.pdf.

[32] Raspberry Pi Ltd, *Raspberry pi 4 model b*, Accessed: 2026-03-01. Archived version used: https://web.archive.org/web/20260305035457/https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008344-DS-2-raspberry-pi-4-product-brief.pdf?disposition=inline, 2025. [Online]. Available: https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008344-DS-2-raspberry-pi-4-product-brief.pdf?disposition=inline.

[33] ifm electronic gmbh, *Operating instructions: Io-link master with iot interface dataline 8 ports (al1352)*, Operating instructions. Available via product page: https://www.ifm.com/us/en/product/AL1352, ifm electronic gmbh, 2020. [Online]. Available: https://www.ifm.com/us/en/product/AL1352.

[34] ifm electronic GmbH, *Moneo configure free – parameter setting software for io-link devices*, Accessed: 2026-04-24. Archived version used: https://web.archive.org/web/20250219215852/https://www.ifm.com/gb/en/shared/productnews/moneo-configure-free, 2024. [Online]. Available: https://www.ifm.com/gb/en/shared/productnews/moneo-configure-free.

[35] D. Spinellis, "Git," *IEEE Software*, vol. 29, no. 3, pp. 100–101, 2012. DOI: 10.1109/MS.2012.61.

[36] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 3rd European ed. London: McGraw-Hill, 1994, Adapted by Darrel Ince, ISBN: 9780077079369.

[37] J. Jason Long Ian Ward, *Json lines documentation for the json lines text file format*, Accessed: 2026-05-08. Archived version used: https://web.archive.org/web/20260529105337/https://jsonlines.org/, 2025. [Online]. Available: https://jsonlines.org/.

[38] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*, Accessed: 2026-06-03 Archived version used: https://web.archive.org/web/20260212020700/https://eur-lex.europa.eu/legal-content/en/TXT/?uri=CELEX%3A32016R0679, 2016. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679.

# Appendix A

# Annex

## A.1 Archived web sources

### A.1.1 VTech Peek Learn Doll Productpage

- Skip to main content
- Skip to search



This site uses cookies to store information on your computer.

Some of these cookies are essential, while others help us to improve your experience by providing insights into how the site is being used. For more detailed information on the cookies we use, please see our Cookie Policy.

I Accept Cookie settings





1. Home
2. >
3. Baby Amaze™ Peek & Learn Doll™











Please read the updated privacy policy. Your use of this site and service constitutes your acceptence of this privacy policy.

Age: 2-5 years

Where to Buy

1. Product Details 2. Product Support 3. Recommended Products

# Product Details

Best for ages:
2 to 5 years
Play peek-a-boo with the Baby Amaze™ Peek & Learn Doll™ by VTech® for adorable reactions and interactive play. Cover the baby doll's eyes with its hands to play the classic game. Press the four interactive buttons on the baby doll and your child will learn body parts, counting and nurturing concepts. The baby doll plays a variety of familiar melodies on a timer for up to 15 minutes. Your little one can also use the included baby bottle for role-play fun. Explore and learn with a cute, interactive baby doll!
Product Number: 80-169400
Battery: 2 AA Batteries Required

Products
Learning Resources
Downloads
Support

## Product Support

- Peek & Learn Doll Manual

# Recommended Products



**Baby**



**KidiBeats**



**Care for Me**



**Drill**

See our Privacy Policy to learn about personal information we collect on this site.
© 2026 VTech Electronics North America, LLC, All Rights Reserved.

Please read the updated privacy policy. Your use of this site and service constitutes your acceptence of this privacy policy.